Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Workspace: E:\ohada saas\Focused projects\stoquify

1. Mission

Audit, redesign, and progressively rewrite the codebase into a substantially smaller, clearer, faster, and more secure implementation while preserving required product capabilities, business rules, data integrity, and externally consumed contracts.

Treat the entire application as eligible for redesign. Justify what to retain, consolidate, replace, or delete using repository evidence.

Pursue aggressive simplification through:

- Removing proven dead code, duplicate implementations, obsolete compatibility layers, and unnecessary dependencies.
- Consolidating repeated business logic under explicit domain ownership.
- Reducing indirection, redundant state, excessive configuration, and unnecessary architectural layers.
- Improving database access, server/client boundaries, rendering, caching, and resource usage where measurements demonstrate a bottleneck.
- Making critical workflows and security controls easier to understand and verify.

Optimize in this order: correctness and security; architectural clarity and maintainability; measured performance and operating cost; code size.

Do not stop at recommendations: produce and verify the implementation in controlled, reviewable stages.

2. Evidence to inspect

Read applicable AGENTS.md instructions, package manifests, lockfiles, build configuration, CI checks, deployment configuration, and existing tests.

Inspect:

- app/, actions/, services/, components/, hooks/, lib/, config/
- prisma/ and migration history

- scripts/ and relevant test directories
- Relevant what-next/ and innovation/ reports
- graphify-out/ architecture graphs and reports

Use graph nodes and communities to identify dependency clusters and change impact. Verify their conclusions against current source code because graphs may be stale.

Trace representative critical workflows from UI through authorization, business logic, persistence, events, external providers, and audit evidence.

Inventory required behavior before removing or replacing its implementation. Static “unused” results alone are insufficient evidence for deleting dynamically loaded routes, jobs, integrations, or configuration.

3. Reviewer coverage

Produce a coverage matrix with an evidence-backed finding or a short “not applicable” reason for every role:

Core: enterprise/platform architecture; backend/domain/integration engineering; database/migration architecture; application security/IAM/privacy/abuse resistance; frontend/design systems; workflow UX/accessibility/localization; product/business processes; quality/release assurance; SRE/DevSecOps/performance; SaaS packaging/billing/growth/customer operations.

Domain: finance/accounting/treasury/reconciliation/internal controls; OHADA/SYSCOHADA/statutory/country-pack compliance; audit/records/data quality; POS/inventory/offline consistency; purchasing/AP/payment controls; HRIS/payroll/employee privacy; payments/mobile money/settlement; accounting close/fiscal reporting/accountant portal; analytics/metric governance; AI/agent safety; API/webhook/outbox/import/export boundaries; change management/documentation/support/rollout.

Use these as review responsibilities; avoid multiplying services or architectural layers to match the reviewer roster.

4. Baselines and measurable targets

Before changing code, record a reproducible baseline and set numerical targets supported by the findings.

Measure:

- Size: maintained application source lines, file count, production dependencies, client bundle size, and deployment artifact size.
- Complexity: duplicated logic, dependency cycles, high-complexity functions, and layers traversed by critical workflows.
- Performance: representative p50/p95 latency, throughput, database query count/time, frontend responsiveness, and memory consumption.
- Reliability and security: critical workflow coverage, existing failures, authorization boundaries, and verified security findings.

Report tests, generated files, vendored code, migrations, documentation, and build output separately. Use consistent counting rules before and after.

Specify workload, dataset size, environment, concurrency, cache state, repeated runs, and acceptable regression thresholds for benchmarks.

Do not invent a reduction percentage before discovery. State the largest defensible target, its tradeoffs, and which measurements remain unavailable.

Never achieve apparent reductions by deleting required behavior, tests, controls, or audit evidence; compressing readable code; moving complexity into opaque helpers; or concealing it inside new dependencies.

5. Execution checklist

A. Inventory behavior and architecture. Identify authoritative business rules, public contracts, consumers, entry points, domain owners, deployment boundaries, and the highest-cost sources of complexity.

B. Design the minimum sufficient architecture. Describe the target structure, ownership boundaries, dependency direction, and migration sequence. Explain how each structural change reduces measured complexity or cost.

Prefer straightforward code and existing platform capabilities. Introduce abstractions only when they simplify demonstrated repetition or enforce a necessary boundary.

C. Establish behavioral protection. Add focused characterization, integration, and contract tests where critical behavior lacks coverage. Cover authorization failures, concurrency, idempotency, retries, and financial invariants where relevant.

Distinguish intended behavior from confirmed defects; give defect corrections explicit regression coverage.

D. Rewrite one representative vertical slice. Carry it through UI, service logic, persistence, and integrations as applicable. Verify behavior, readability, performance, and security against the baseline before applying the design broadly.

E. Complete the remaining scope in dependency order. Keep each stage runnable and reviewable. Remove superseded implementations once replacements and consumers are verified. Track temporary adapters with explicit removal conditions.

F. Validate the integrated result. Run cross-module workflows and release checks. Report remaining legacy paths, regressions, blockers, and unmet targets honestly. A completed pilot is not a completed codebase rewrite.

6. Risk controls

Preserve:

- Tenant isolation, server-side authorization, RBAC, module entitlements, and least privilege.
- Service-owned business truth, transaction boundaries, monetary precision, idempotency, and concurrency protection.
- Segregation of duties, approval history, immutable audit evidence, redaction, and secrets protection.
- Accounting, inventory, payroll, reconciliation, and close invariants wherever affected.
- API/event contracts, offline replay behavior, recovery paths, accessibility, and localization.

Separate statutory and country-specific rules from generic application logic. Preserve dated provenance and required expert review.

Use compatible, reversible migration stages with data reconciliation and tested recovery procedures. Do not use destructive resets as a migration strategy.

Inspect the dirty worktree first. Preserve unrelated user changes and do not touch unrelated lint warnings.

7. Verification commands and evidence

Discover the package manager and actual scripts from repository files before selecting commands. Record the exact commands used.

Where these scripts exist and apply, run:

- npm run typecheck
- npm run policy:gates
- npm run workflow:assurance:runtime-check
- npm run prisma:validate
- npm run build:app

Adapt invocation to the repository's package manager. Add focused unit, integration, contract, migration, browser, accessibility, and benchmark commands according to the changed boundaries.

Exercise negative authorization cases, cross-tenant access attempts, failure recovery, and critical business invariants.

Compare baseline and replacement using identical measurement conditions. Record passed, failed, skipped, and blocked checks, including pre-existing failures. Never describe an unexecuted check as passing.

8. Expected artifacts

Save under what-next/codebase-rewrite/:

- baseline.md: architecture inventory, behavior contracts, measurements, and known failures.
- rewrite-plan.md: target architecture, numerical targets, dependency order, tradeoffs, and migration/rollback strategy.
- progress.md: module-by-module retained, replaced, removed, and outstanding work.
- verification.md: reproducible commands, results, benchmark comparisons, and reviewer coverage.
- final-report.md: achieved reductions, performance changes, security findings addressed, remaining risks, and unmet targets.

Deliver the implementation, necessary tests, updated architecture documentation, and removal of verified obsolete code.

9. Success criteria

The rewrite is complete when:

- All inventoried required workflows pass their applicable checks.
- Critical security and data-integrity invariants remain enforced.
- The codebase meets its evidence-backed reduction and complexity targets, or explicitly documents any unmet target.
- Performance improvements are reproduced and critical workflows stay within declared regression budgets.
- Superseded implementations and temporary migration layers are removed or explicitly accounted for.
- Another engineer can trace critical workflows and understand ownership with fewer conceptual steps.
- No claimed result relies on hidden scope exclusions or unverified assumptions.

10. Non-goals

Do not add speculative features, redesign product behavior for aesthetic preference, change frameworks without demonstrated benefit, remove required capabilities to reduce size, or deploy to production as part of this task.